

Name: Rushang Bagada
RollNo:U24CS076

```
// Q1: Student Details with iomanip Manipulators
// Demonstrates: setw, setprecision, left, right, fixed, getline(), get()

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Student {
    int rollNo;
    string name;
    float marks[3];
    float total;
    float average;
};

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    cin.ignore(); // Clear newline from buffer

    Student students[n];

    // Input student details
    for(int i = 0; i < n; i++) {
        cout << "\n--- Student " << (i+1) << " ---\n";

        cout << "Enter Roll Number: ";
        cin >> students[i].rollNo;
        cin.ignore();

        cout << "Enter Name: ";
        getline(cin, students[i].name);

        students[i].total = 0;
```

```

        for(int j = 0; j < 3; j++) {
            cout << "Enter marks for Subject " << (j+1) << ": ";
            cin >> students[i].marks[j];
            students[i].total += students[i].marks[j];
        }
        cin.ignore();

        students[i].average = students[i].total / 3.0;
    }

    // Display formatted table
    cout << "\n\n";
    cout << string(90, '=') << endl;
    cout << setw(45) << "STUDENT REPORT CARD" << endl;
    cout << string(90, '=') << endl;

    // Table header
    cout << left << setw(10) << "Roll No"
        << setw(25) << "Name"
        << right << setw(12) << "Subject 1"
        << setw(12) << "Subject 2"
        << setw(12) << "Subject 3"
        << setw(10) << "Total"
        << setw(10) << "Average" << endl;
    cout << string(90, '-') << endl;

    // Display student data
    for(int i = 0; i < n; i++) {
        cout << left << setw(10) << students[i].rollNo
            << setw(25) << students[i].name;

        for(int j = 0; j < 3; j++) {
            cout << right << setw(12) << fixed << setprecision(2)
                << students[i].marks[j];
        }

        cout << setw(10) << fixed << setprecision(2) << students[i].total
            << setw(10) << fixed << setprecision(2) <<
students[i].average << endl;
    }

```

```

    cout << string(90, '=') << endl;

    return 0;
}

```

```

Enter number of students: 2

--- Student 1 ---
Enter Roll Number: 76
Enter Name: rushang
Enter marks for Subject 1: 70
Enter marks for Subject 2: 70
Enter marks for Subject 3: 70

--- Student 2 ---
Enter Roll Number: 22
Enter Name: rudraksh
Enter marks for Subject 1: 80
Enter marks for Subject 2: 80
Enter marks for Subject 3: 80

```

```

=====
                        STUDENT REPORT CARD
=====
Roll No   Name                Subject 1   Subject 2   Subject 3   Total   Average
-----
76        rushang                70.00      70.00      70.00      210.00   70.00
22        rudraksh                80.00      80.00      80.00      240.00   80.00
=====

```

```

// Q2: Function Template for Statistical Analysis
// Demonstrates: Function templates, array operations, iomanip

#include <iostream>
#include <iomanip>
#include <algorithm>
#include <cmath>
using namespace std;

template <typename T>
void analyze(T arr[], int n) {
    // Calculate Mean
    T sum = 0;
    for(int i = 0; i < n; i++) {

```

```

        sum += arr[i];
    }
    double mean = static_cast<double>(sum) / n;

    // Calculate Median
    T temp[n];
    for(int i = 0; i < n; i++) {
        temp[i] = arr[i];
    }
    sort(temp, temp + n);

    double median;
    if(n % 2 == 0) {
        median = (temp[n/2 - 1] + temp[n/2]) / 2.0;
    } else {
        median = temp[n/2];
    }

    // Calculate Standard Deviation
    double variance = 0;
    for(int i = 0; i < n; i++) {
        variance += pow(arr[i] - mean, 2);
    }
    variance /= n;
    double stdDev = sqrt(variance);

    // Display results
    cout << fixed << setprecision(2);
    cout << "\n--- Statistical Analysis ---\n";
    cout << left << setw(25) << "Mean: " << right << setw(10) << mean <<
endl;
    cout << left << setw(25) << "Median: " << right << setw(10) << median
<< endl;
    cout << left << setw(25) << "Standard Deviation: " << right <<
setw(10) << stdDev << endl;
    cout << string(35, '-') << endl;
}

int main() {
    int choice;

```

```
cout << "Select data type:\n";
cout << "1. Integer\n";
cout << "2. Float\n";
cout << "3. Double\n";
cout << "Enter choice: ";
cin >> choice;

int n;
cout << "Enter number of elements: ";
cin >> n;

switch(choice) {
    case 1: {
        int arr[n];
        cout << "Enter " << n << " integer values:\n";
        for(int i = 0; i < n; i++) {
            cout << "Element " << (i+1) << ": ";
            cin >> arr[i];
        }
        analyze(arr, n);
        break;
    }
    case 2: {
        float arr[n];
        cout << "Enter " << n << " float values:\n";
        for(int i = 0; i < n; i++) {
            cout << "Element " << (i+1) << ": ";
            cin >> arr[i];
        }
        analyze(arr, n);
        break;
    }
    case 3: {
        double arr[n];
        cout << "Enter " << n << " double values:\n";
        for(int i = 0; i < n; i++) {
            cout << "Element " << (i+1) << ": ";
            cin >> arr[i];
        }
        analyze(arr, n);
    }
}
```

```
        break;
    }
    default:
        cout << "Invalid choice!\n";
    }

    // Confirmation prompt using iostream functions
    char confirm;
    cout << "\nWant to analyze another dataset? (y/n): ";
    cin.get(); // Clear buffer
    confirm = cin.get();

    if(confirm == 'y' || confirm == 'Y') {
        cout << "Rerun the program!\n";
    } else {
        cout << "Thank you!\n";
    }

    return 0;
}
```

```
Select data type:
1. Integer
1. Integer
2. Float
3. Double
Enter choice: 1
Enter number of elements: 2
Enter 2 integer values:
Element 1: 11
Element 2: 22

--- Statistical Analysis ---
Mean:                16.50
Median:              16.50
Standard Deviation:  5.50
-----
```

```
// Q3: Mini Bank Statement Generator
// Demonstrates: fixed, setprecision, setw, currency formatting

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    string name, accountNumber;
    double balance;

    cout << "=== Bank Statement Generator ===\n\n";

    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, name);
```

```

cout << "Enter Account Number: ";
getline(cin, accountNumber);

cout << "Enter Balance Amount: ";
cin >> balance;

// Display Mini Statement
cout << "\n\n";
cout << string(60, '=') << endl;
cout << setw(35) << "MINI BANK STATEMENT" << endl;
cout << string(60, '=') << endl;

cout << left << setw(20) << "Customer Name:"
    << setw(40) << name << endl;

cout << left << setw(20) << "Account Number:"
    << setw(40) << accountNumber << endl;

cout << string(60, '-') << endl;

cout << left << setw(20) << "Current Balance:"
    << right << "₹ " << fixed << setprecision(2)
    << setw(15) << balance << endl;

cout << string(60, '=') << endl;

// Additional transaction display
cout << "\n" << left << setw(30) << "Transaction Type"
    << right << setw(30) << "Amount (₹)" << endl;
cout << string(60, '-') << endl;

cout << left << setw(30) << "Opening Balance"
    << right << "₹ " << fixed << setprecision(2)
    << setw(15) << balance << endl;

cout << left << setw(30) << "Interest Credited"
    << right << "₹ " << fixed << setprecision(2)
    << setw(15) << (balance * 0.04) << endl;

```

```

double closingBalance = balance + (balance * 0.04);
cout << string(60, '-') << endl;
cout << left << setw(30) << "Closing Balance"
    << right << "₹ " << fixed << setprecision(2)
    << setw(15) << closingBalance << endl;

cout << string(60, '=') << endl;
cout << "\nThank you for banking with us!\n";

return 0;
}

```

=== Bank Statement Generator ===

Enter Customer Name: Rushang
Enter Account Number: 76
Enter Balance Amount: 5000000

=====

MINI BANK STATEMENT

=====

Customer Name: ushang
Account Number: 76

Current Balance: ₹ 5000000.00

=====

Transaction Type	Amount (₹)
------------------	------------

Opening Balance	₹	5000000.00
Interest Credited	₹	200000.00

Closing Balance	₹	5200000.00
-----------------	---	------------

=====

Thank you for banking with us!

```

// Q4: Product Bill Generator
// Demonstrates: setw, fixed, setprecision, width(), fill(), precision()

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Product {
    string name;
    int quantity;
    double pricePerItem;
    double total;
};

int main() {
    int n;
    cout << "Enter number of products: ";
    cin >> n;
    cin.ignore();

    Product products[n];
    double grandTotal = 0;

    // Input product details
    for(int i = 0; i < n; i++) {
        cout << "\n--- Product " << (i+1) << " ---\n";
        cout << "Enter Product Name: ";
        getline(cin, products[i].name);

        cout << "Enter Quantity: ";
        cin >> products[i].quantity;

        cout << "Enter Price per Item: ";
        cin >> products[i].pricePerItem;
        cin.ignore();

        products[i].total = products[i].quantity *
products[i].pricePerItem;

```

```

        grandTotal += products[i].total;
    }

    // Display Bill
    cout << "\n\n";
    cout.fill('=');
    cout.width(80);
    cout << "" << endl;
    cout.fill(' ');

    cout << setw(45) << "INVOICE / BILL" << endl;

    cout.fill('=');
    cout.width(80);
    cout << "" << endl;
    cout.fill(' ');

    // Table Header
    cout << left << setw(5) << "No."
        << setw(30) << "Product Name"
        << right << setw(15) << "Quantity"
        << setw(15) << "Price/Item"
        << setw(15) << "Total" << endl;

    cout.fill('-');
    cout.width(80);
    cout << "" << endl;
    cout.fill(' ');

    // Product rows
    for(int i = 0; i < n; i++) {
        cout << left << setw(5) << (i+1)
            << setw(30) << products[i].name;

        cout.precision(0);
        cout << right << setw(15) << products[i].quantity;

        cout.precision(2);
        cout << fixed << right << setw(15) << products[i].pricePerItem
            << setw(15) << products[i].total << endl;
    }

```

```

}

cout.fill('-');
cout.width(80);
cout << "" << endl;
cout.fill(' ');

// Grand Total
cout << left << setw(50) << ""
    << right << setw(15) << "Grand Total:"
    << fixed << setprecision(2) << setw(15) << grandTotal << endl;

// Tax calculation
double tax = grandTotal * 0.18; // 18% GST
cout << left << setw(50) << ""
    << right << setw(15) << "Tax (18%):"
    << fixed << setprecision(2) << setw(15) << tax << endl;

double finalAmount = grandTotal + tax;
cout.fill('=');
cout.width(80);
cout << "" << endl;
cout.fill(' ');

cout << left << setw(50) << ""
    << right << setw(15) << "Final Amount:"
    << fixed << setprecision(2) << setw(15) << finalAmount << endl;

cout.fill('=');
cout.width(80);
cout << "" << endl;
cout.fill(' ');

cout << "\nThank you for your purchase!\n";

return 0;
}

```

```

Enter number of products: 2

--- Product 1 ---
Enter Product Name: toolkit
Enter Quantity: 3
Enter Price per Item: 50

--- Product 2 ---
Enter Product Name: lapto[
Enter Quantity: 3
Enter Price per Item: 120000

=====
                        INVOICE / BILL
=====
No.   Product Name                Quantity   Price/Item   Total
-----
1     toolkit                      3           50.00       150.00
2     lapto[                      3        120000.00   360000.00
-----
                        Grand Total:      360150.00
                        Tax (18%):        64827.00
=====
                        Final Amount:      424977.00
=====

Thank you for your purchase!

```

```

// Q5: Template-Based Statistics Analyzer
// Demonstrates: Class templates, function templates, manipulators

#include <iostream>
#include <iomanip>
#include <algorithm>
using namespace std;

// Class template for Statistics
template <typename T>
class Statistics {
private:
    T* arr;
    int size;

public:

```

```

Statistics(T a[], int n) : size(n) {
    arr = new T[size];
    for(int i = 0; i < size; i++) {
        arr[i] = a[i];
    }
}

~Statistics() {
    delete[] arr;
}

T sum() {
    T total = 0;
    for(int i = 0; i < size; i++) {
        total += arr[i];
    }
    return total;
}

double average() {
    return static_cast<double>(sum()) / size;
}

T maximum() {
    T max = arr[0];
    for(int i = 1; i < size; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

void display() {
    cout << fixed << setprecision(2);
    cout << left << setw(20) << "Sum = " << sum() << endl;
    cout << left << setw(20) << "Average = " << average() << endl;
    cout << left << setw(20) << "Maximum = " << maximum() << endl;
}
};

```

```

// Function template to compare two values
template <typename T>
T compareValues(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    // Integer array (student scores)
    int intArr[] = {78, 65, 92, 55};
    int intSize = sizeof(intArr) / sizeof(intArr[0]);

    cout << "Integer Data (Student Scores):\n";
    cout << string(40, '-') << endl;
    Statistics<int> intStats(intArr, intSize);
    intStats.display();
    cout << endl;

    // Double array (product prices)
    double doubleArr[] = {199.99, 49.50, 274.26, 499.75};
    int doubleSize = sizeof(doubleArr) / sizeof(doubleArr[0]);

    cout << "Double Data (Product Prices):\n";
    cout << string(40, '-') << endl;
    Statistics<double> doubleStats(doubleArr, doubleSize);
    doubleStats.display();
    cout << endl;

    // Comparing values
    int val1 = 25, val2 = 42;
    cout << "Comparing " << val1 << " and " << val2
        << " → Greater value: " << compareValues(val1, val2) << endl;

    double dval1 = 45.67, dval2 = 32.89;
    cout << fixed << setprecision(2);
    cout << "Comparing " << dval1 << " and " << dval2
        << " → Greater value: " << compareValues(dval1, dval2) << endl;

    return 0;
}

```

Integer Data (Student Scores):

```
-----  
Sum =                290  
Average =            72.50  
Maximum =            92
```

Double Data (Product Prices):

```
-----  
Sum =                1023.50  
Average =            255.88  
Maximum =            499.75
```

Comparing 25 and 42 → Greater value: 42

Comparing 45.67 and 32.89 → Greater value: 45.67

```
// Q6: Employee Payroll Slip Generator with Stream Flags  
// Demonstrates: showpos, internal, setfill, setw, stream flags  
  
#include <iostream>  
#include <iomanip>  
#include <string>  
using namespace std;  
  
struct Employee {  
    int empID;  
    string name;  
    double hourlyWage;  
    double hoursWorked;  
    double grossPay;  
};  
  
int main() {  
    int n;  
    double threshold = 500.0; // Minimum pay threshold
```

```

cout << "Enter number of employees: ";
cin >> n;
cin.ignore();

Employee employees[n];

// Input employee details
for(int i = 0; i < n; i++) {
    cout << "\n--- Employee " << (i+1) << " ---\n";

    cout << "Enter Employee ID: ";
    cin >> employees[i].empID;
    cin.ignore();

    cout << "Enter Name: ";
    getline(cin, employees[i].name);

    cout << "Enter Hourly Wage: ";
    cin >> employees[i].hourlyWage;

    cout << "Enter Hours Worked: ";
    cin >> employees[i].hoursWorked;
    cin.ignore();

    employees[i].grossPay = employees[i].hourlyWage *
employees[i].hoursWorked;
}

// Display Payroll Slips
cout << "\n\n";
cout << string(90, '=') << endl;
cout << setw(50) << "PAYROLL SLIP REPORT" << endl;
cout << string(90, '=') << endl;

// Table Header
cout << left << setw(30) << "Employee Name"
    << right << setw(15) << "Employee ID"
    << setw(15) << "Hourly Wage"
    << setw(15) << "Hours"

```

```

        << setw(15) << "Gross Pay" << endl;
    cout << string(90, '-') << endl;

    // Display each employee's payroll
    for(int i = 0; i < n; i++) {
        // Name - left aligned
        cout << left << setw(30) << employees[i].name;

        // ID - right aligned, padded with zeros, width 8
        cout << right << setfill('0') << setw(15) << employees[i].empID;
        cout << setfill(' ');

        // Hourly wage - with sign
        cout << showpos << fixed << setprecision(2)
            << setw(15) << employees[i].hourlyWage;

        // Hours worked
        cout << noshowpos << fixed << setprecision(2)
            << setw(15) << employees[i].hoursWorked;

        // Gross pay - with sign and internal alignment
        cout << showpos << internal << setfill('*')
            << setw(15) << fixed << setprecision(2)
            << employees[i].grossPay << noshowpos << setfill(' ');

        cout << endl;

        // Warning if below threshold
        if(employees[i].grossPay < threshold) {
            cout << "⚠ WARNING: Gross pay for " << employees[i].name
                << " is below threshold!" << endl << flush;
        }
    }

    cout << string(90, '=') << endl;

    // Individual Payroll Slips
    for(int i = 0; i < n; i++) {
        cout << "\n\n";
        cout << string(60, '=') << endl;
    }

```

```

cout << setw(35) << "PAYROLL SLIP" << endl;
cout << string(60, '=') << endl;

cout << left << setw(20) << "Employee Name:"
    << setw(40) << employees[i].name << endl;

cout << left << setw(20) << "Employee ID:"
    << right << setfill('0') << setw(8) << employees[i].empID
    << setfill(' ') << endl;

cout << string(60, '-') << endl;

cout << left << setw(30) << "Hourly Wage:"
    << right << showpos << "$ " << fixed << setprecision(2)
    << setw(10) << employees[i].hourlyWage << noshowpos << endl;

cout << left << setw(30) << "Hours Worked:"
    << right << fixed << setprecision(2)
    << setw(12) << employees[i].hoursWorked << endl;

cout << string(60, '-') << endl;

cout << left << setw(30) << "Gross Pay:"
    << right << showpos << "$ " << fixed << setprecision(2)
    << setw(10) << employees[i].grossPay << noshowpos << endl;

cout << string(60, '=') << endl;

if(employees[i].grossPay < threshold) {
    cout << "\n⚠ WARNING: Payment below minimum threshold of $"
        << fixed << setprecision(2) << threshold << endl <<
flush;
}

}

cout << "\n\nPayroll processing completed!" << endl;

return 0;
}

```

--- Employee 1 ---
Enter Employee ID: 1
Enter Name: rudi
Enter Hourly Wage: 5
Enter Hours Worked: 6

--- Employee 2 ---
Enter Employee ID: 2
Enter Name: gaurav
Enter Hourly Wage:
9
Enter Hours Worked: 9

=====

PAYROLL SLIP REPORT

=====

Employee Name	Employee ID	Hourly Wage	Hours	Gross Pay
rudi	0000000000000001	+5.00	6.00	+*****30.00
⚠ WARNING: Gross pay for rudi is below threshold!				
gaurav	0000000000000002	+9.00	9.00	+*****81.00
⚠ WARNING: Gross pay for gaurav is below threshold!				

=====

=====

PAYROLL SLIP

=====

Employee Name: rudi
Employee ID: 00000001

Hourly Wage: \$ +5.00
Hours Worked: 6.00

Gross Pay: \$ +30.00

=====

⚠ WARNING: Payment below minimum threshold of \$500.00

=====

PAYROLL SLIP

=====

Employee Name: gaurav
Employee ID: 00000002

Hourly Wage: \$ +9.00
Hours Worked: 9.00

Gross Pay: \$ +81.00

=====

```

// Demonstrates: Class templates, custom fill, table formatting

#include <iostream>
#include <iomanip>
using namespace std;

template <typename T>
class CircularBuffer {
private:
    T* buffer;
    int capacity;
    int head;
    int tail;
    int count;

public:
    CircularBuffer(int size) : capacity(size), head(0), tail(0), count(0)
    {
        buffer = new T[capacity];
    }

    ~CircularBuffer() {
        delete[] buffer;
    }

    bool insert(T element) {
        if(count == capacity) {
            cout << "Buffer full! Overwriting oldest element.\n";
            head = (head + 1) % capacity;
            count--;
        }

        buffer[tail] = element;
        tail = (tail + 1) % capacity;
        count++;
        return true;
    }

    bool remove(T& element) {
        if(count == 0) {

```

```

        cout << "Buffer empty!\n";
        return false;
    }

    element = buffer[head];
    head = (head + 1) % capacity;
    count--;
    return true;
}

void display() {
    cout << "\n";
    cout.fill('=');
    cout.width(70);
    cout << "" << endl;
    cout.fill(' ');

    cout << setw(40) << "CIRCULAR BUFFER CONTENTS" << endl;

    cout.fill('=');
    cout.width(70);
    cout << "" << endl;
    cout.fill(' ');

    cout << left << setw(10) << "Index"
        << setw(15) << "Position"
        << setw(20) << "Value"
        << setw(15) << "Status" << endl;

    cout.fill('-');
    cout.width(70);
    cout << "" << endl;
    cout.fill(' ');

    int pos = head;
    for(int i = 0; i < count; i++) {
        cout << left << setw(10) << i;

        cout << setfill('.') << setw(15) << pos << setfill(' ');
    }
}

```

```

        // Format based on type
        cout << left << setw(20);
        if(typeid(T) == typeid(double) || typeid(T) == typeid(float))
{
            cout << fixed << setprecision(2) << buffer[pos];
        } else if(typeid(T) == typeid(char)) {
            cout << buffer[pos];
        } else {
            cout << buffer[pos];
        }

        if(pos == head && pos == (tail - 1 + capacity) % capacity) {
            cout << setw(15) << "HEAD & TAIL";
        } else if(pos == head) {
            cout << setw(15) << "HEAD";
        } else if(pos == (tail - 1 + capacity) % capacity) {
            cout << setw(15) << "TAIL";
        } else {
            cout << setw(15) << "DATA";
        }

        cout << endl;

        // Insert horizontal line after every 5 elements
        if((i + 1) % 5 == 0 && i < count - 1) {
            cout.fill('~');
            cout.width(70);
            cout << "" << endl;
            cout.fill(' ');
        }

        pos = (pos + 1) % capacity;
    }

    cout.fill('=');
    cout.width(70);
    cout << "" << endl;
    cout.fill(' ');

    cout << "\nBuffer Statistics:\n";

```

```

        cout << "Capacity: " << capacity << " | ";
        cout << "Count: " << count << " | ";
        cout << "Head: " << head << " | ";
        cout << "Tail: " << tail << endl;

        cout.fill('=');
        cout.width(70);
        cout << "" << endl;
        cout.fill(' ');
    }
};

int main() {
    cout << "=== Circular Buffer Demo ===\n\n";

    // Test with char type
    cout << "--- Testing with CHAR type ---\n";
    CircularBuffer<char> charBuffer(8);

    charBuffer.insert('A');
    charBuffer.insert('B');
    charBuffer.insert('C');
    charBuffer.insert('D');
    charBuffer.insert('E');
    charBuffer.insert('F');
    charBuffer.insert('G');

    charBuffer.display();

    char removed;
    charBuffer.remove(removed);
    cout << "\nRemoved: " << removed << endl;
    charBuffer.insert('H');
    charBuffer.insert('I');

    charBuffer.display();

    // Test with double type
    cout << "\n\n--- Testing with DOUBLE type ---\n";
    CircularBuffer<double> doubleBuffer(12);

```

```
doubleBuffer.insert(23.45);
doubleBuffer.insert(67.89);
doubleBuffer.insert(12.34);
doubleBuffer.insert(99.99);
doubleBuffer.insert(45.67);
doubleBuffer.insert(78.90);
doubleBuffer.insert(34.56);
doubleBuffer.insert(89.12);
doubleBuffer.insert(56.78);
doubleBuffer.insert(90.12);

doubleBuffer.display();

double dRemoved;
doubleBuffer.remove(dRemoved);
doubleBuffer.remove(dRemoved);
cout << "\nLast removed: " << fixed << setprecision(2) << dRemoved <<
endl;

doubleBuffer.insert(11.11);
doubleBuffer.insert(22.22);
doubleBuffer.insert(33.33);

doubleBuffer.display();

return 0;
}
```

--- Testing with CHAR type ---

CIRCULAR BUFFER CONTENTS			
Index	Position	Value	Status
0	0.....	A	HEAD
1	1.....	B	DATA
2	2.....	C	DATA
3	3.....	D	DATA
4	4.....	E	DATA
5	5.....	F	DATA
6	6.....	G	TAIL

Buffer Statistics:

Capacity: 8 | Count: 7 | Head: 0 | Tail: 7

Removed: A

CIRCULAR BUFFER CONTENTS			
Index	Position	Value	Status
0	1.....	B	HEAD
1	2.....	C	DATA
2	3.....	D	DATA
3	4.....	E	DATA
4	5.....	F	DATA
5	6.....	G	DATA
6	7.....	H	DATA
7	0.....	I	TAIL

Buffer Statistics:

Capacity: 8 | Count: 8 | Head: 1 | Tail: 1

--- Testing with DOUBLE type ---

CIRCULAR BUFFER CONTENTS			
Index	Position	Value	Status
0	0.....	23.45	HEAD
1	1.....	67.89	DATA
2	2.....	12.34	DATA

```

// Q8: Date and Time Table with User-Defined Manipulator
// Demonstrates: Custom manipulators, date formatting, table display

#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
using namespace std;

// User-defined manipulator for date formatting
ostream& formatDate(ostream& os) {
    // This is a simple manipulator that can be extended
    // For real date formatting, you'd pass date strings through
    return os;
}

struct Event {
    string name;
    string date; // Format: YYYY-MM-DD
};

// Function to validate and format date
string validateDate(const string& date) {
    // Basic validation - assumes format YYYY-MM-DD or variations
    if(date.length() >= 8) {
        // Extract components
        stringstream ss(date);
        string year, month, day;
        char delim;

        if(date.find('-') != string::npos) {
            getline(ss, year, '-');
            getline(ss, month, '-');
            getline(ss, day, '-');
        } else if(date.find('/') != string::npos) {
            getline(ss, year, '/');
            getline(ss, month, '/');
            getline(ss, day, '/');
        } else {
            return date;
        }
    }
}

```

```

    }

    // Ensure proper formatting
    if(month.length() == 1) month = "0" + month;
    if(day.length() == 1) day = "0" + day;

    return year + "-" + month + "-" + day;
}

return date;
}

// Custom manipulator function that formats dates
class DateFormatter {
private:
    string date;
public:
    DateFormatter(const string& d) : date(d) {}

    friend ostream& operator<<(ostream& os, const DateFormatter& df) {
        os << validateDate(df.date);
        return os;
    }
};

DateFormatter formatDateString(const string& date) {
    return DateFormatter(date);
}

int main() {
    int n;
    cout << "Enter number of events: ";
    cin >> n;
    cin.ignore();

    Event events[n];

    // Input event details
    for(int i = 0; i < n; i++) {
        cout << "\n--- Event " << (i+1) << " ---\n";
    }
}

```

```

        cout << "Enter Event Name: ";
        getline(cin, events[i].name);

        cout << "Enter Date (YYYY-MM-DD): ";
        getline(cin, events[i].date);

        // Validate and format date
        events[i].date = validateDate(events[i].date);
    }

    // Display Events Table
    cout << "\n\n";
    cout << string(80, '=') << endl;
    cout << setw(45) << "EVENT SCHEDULE" << endl;
    cout << string(80, '=') << endl;

    // Table Header
    cout << left << setw(10) << "Sr. No."
        << setw(45) << "Event Name"
        << right << setw(25) << "Date (YYYY-MM-DD)" << endl;
    cout << string(80, '-') << endl;

    // Display events
    for(int i = 0; i < n; i++) {
        cout << left << setw(10) << (i+1)
            << setw(45) << events[i].name
            << right << setw(25) << formatDateString(events[i].date) <<
endl;
    }

    cout << string(80, '=') << endl;

    // Display in alternative format
    cout << "\n\n";
    cout << string(80, '=') << endl;
    cout << setw(50) << "DETAILED EVENT LIST" << endl;
    cout << string(80, '=') << endl;

    for(int i = 0; i < n; i++) {
        cout << "\n";

```

```

    cout << right << setw(15) << "Event #" << (i+1) << endl;
    cout << string(80, '-') << endl;

    cout << left << setw(20) << "Event Name:"
        << setw(60) << events[i].name << endl;

    cout << left << setw(20) << "Scheduled Date:"
        << right << formatDateString(events[i].date) << endl;

    // Parse and display date components
    stringstream ss(events[i].date);
    string year, month, day;
    getline(ss, year, '-');
    getline(ss, month, '-');
    getline(ss, day, '-');

    cout << left << setw(20) << "  Year:" << year << endl;
    cout << left << setw(20) << "  Month:" << month << endl;
    cout << left << setw(20) << "  Day:" << day << endl;
}

cout << "\n" << string(80, '=') << endl;

// Calendar view
cout << "\n\n";
cout << string(80, '=') << endl;
cout << setw(45) << "CALENDAR VIEW" << endl;
cout << string(80, '=') << endl;

cout << right << setw(25) << "Date"
    << setw(55) << "Event" << endl;
cout << string(80, '-') << endl;

// Sort events by date (simple bubble sort)
for(int i = 0; i < n-1; i++) {
    for(int j = 0; j < n-i-1; j++) {
        if(events[j].date > events[j+1].date) {
            Event temp = events[j];
            events[j] = events[j+1];
            events[j+1] = temp;
        }
    }
}

```

```
    }  
    }  
}  
  
// Display sorted events  
for(int i = 0; i < n; i++) {  
    cout << right << setw(25) << formatDateString(events[i].date)  
        << " → " << left << events[i].name << endl;  
}  
  
cout << string(80, '=') << endl;  
  
return 0;  
}
```

--- Event 1 ---
Enter Event Name: kashish
Enter Date (YYYY-MM-DD): 2025-11-15

--- Event 2 ---
Enter Event Name: mindbend
Enter Date (YYYY-MM-DD): 2025-03-07

=====

EVENT SCHEDULE

=====

Sr. No.	Event Name	Date (YYYY-MM-DD)
1	kashish	2025-11-15
2	mindbend	2025-03-07

=====

=====

DETAILED EVENT LIST

=====

Event #1

Event Name: kashish
Scheduled Date: 2025-11-15
Year: 2025
Month: 11
Day: 15

Event #2

Event Name: mindbend
Scheduled Date: 2025-03-07
Year: 2025
Month: 03
Day: 07

=====